

VirusSimulation

Symulacja epidemii w konsoli

— Dokumentacja —

Spis treści

1	Wstęp	3
1.1	Cel projektu	3
1.2	Wymagania techniczne	3
1.3	Funkcjonalności	3
2	Struktura projektu	3
2.1	Układ katalogów	3
2.2	Pliki konfiguracyjne	4
3	Architektura i wzorce projektowe	4
3.1	Hierarchia dziedziczenia	4
3.2	Zastosowane wzorce projektowe	5
3.2.1	Wzorzec Strategii (Strategy)	5
3.2.2	Metoda Szablonowa (Template Method)	5
3.2.3	Wzorzec Fabryki (Factory)	5
3.2.4	Wstrzykiwanie Zależności (Dependency Injection)	5
3.2.5	Segregacja Interfejsów (Interface Segregation)	6
3.3	Pętla symulacji	6
4	Opis komponentów	6
4.1	Interfejsy (Interfaces/)	6
4.2	Jednostki (Models/Entities/)	7
4.2.1	Entity	7
4.2.2	Hospital	7
4.3	Ludzie (Models/Humans/)	7
4.3.1	Human	7
4.3.2	Worker	8
4.3.3	Student	8
4.3.4	Elder	8
4.3.5	Doctor	8
4.4	Wirusy (Models/Viruses/)	8
4.5	Strategie rozprzestrzeniania (Models/SpreadLogic/)	9
4.5.1	SpreadLogicBase	9
4.5.2	UniformSpreadLogic	9
4.5.3	DistanceWeightedSpreadLogic	9
4.5.4	FocusedSpreadLogic	9
4.6	Plansza i generowanie mapy	9
4.6.1	Board	9
4.6.2	MapGenerator	10
4.7	Renderowanie	10
4.8	Statystyki	10
5	Mechanizmy symulacyjne	11
5.1	Świadomość społeczna (Awareness)	11
5.2	Blokada (Lockdown)	11
5.3	System leczenia	11

6 Testy	11
6.1 Konfiguracja	11
6.2 Pokrycie testów	12
6.3 Uruchamianie	12
7 Instrukcja uruchomienia	12
7.1 Wymagania	12
7.2 Budowanie	12
7.3 Uruchamianie	12
7.4 Przebieg symulacji	12
8 Podsumowanie	13
A Diagramy UML	14
A.1 Diagram klas	14
A.2 Diagram obiektów	14
A.3 Diagram sekwencji	15
A.4 Diagram stanów	16

1. Wstęp

1.1. Cel projektu

Celem projektu jest stworzenie konsolowej symulacji rozprzestrzeniania się wirusa na dwuwymiarowej, proceduralnie generowanej mapie. Symulacja uwzględnia różne typy ludności (pracownicy, studenci, seniorzy, lekarze), wirusy o zróżnicowanych parametrach oraz mechanizmy zapobiegawcze, takie jak blokada regionów i świadomość społeczna.

1.2. Wymagania techniczne

- **Język:** C# 13.0
- **Platforma:** .NET 10.0
- **Testy:** xUnit 2.x
- **System kontroli wersji:** Git
- **CI:** GitHub Actions (sprawdzanie formatowania kodu)

1.3. Funkcjonalności

- Proceduralne generowanie mapy z czterema regionami oddzielonymi wodą
- Trzy typy wirusów o różnych parametrach: Flu, Covid, Rabies
- Pięć typów jednostek: Worker, Student, Elder, Doctor, Hospital
- Trzy strategie rozprzestrzeniania się wirusa: jednolita, odległościowa, skupiona
- System leczenia (lekarze mobilni i szpitale stacjonarne)
- Mechanizmy zapobiegawcze: blokada regionów, świadomość społeczna
- Migracja ludności między regionami
- Eksport statystyk do pliku CSV
- Wizualizacja w konsoli z kolorami ANSI

2. Struktura projektu

2.1. Układ katalogów

```

VirusSimulation/
  Interfaces/      - Interfejsy (ITile, IVirus, ISpreadLogic,
                    IHealingAbility, IRandom)
  Models/
    Entities/      - Entity (bazowa), Hospital
    Humans/        - Human (bazowa), Worker, Student, Elder,
    Doctor
    Map/           - Tile (enum)
    SpreadLogic/   - Strategie rozprzestrzeniania
    Viruses/       - Flu, Covid, Rabies
  Utilities/       - Board, MapGenerator, Rendering, Statistics
    , GameRandom
  tests/           - Projekt testowy xUnit
  *.puml           - Diagramy PlantUML (klas, obiektów,
                    sekwencji, stanów)

```

2.2. Pliki konfiguracyjne

- VirusSimulation.csproj — główny plik projektu (.NET 10.0, exe)
- tests/VirusSimulation.Tests/VirusSimulation.Tests.csproj — projekt testowy
- tests/VirusSimulation.Tests/xunit.runner.json — konfiguracja testów
- .github/workflows/dotnet-format.yml — CI w GitHub Actions

3. Architektura i wzorce projektowe

Projekt został zaprojektowany z zachowaniem zasad programowania obiektowego i wykorzystuje kilka kluczowych wzorców projektowych.

3.1. Hierarchia dziedziczenia

Rysunek 1 przedstawia pełny diagram klas. Główna hierarchia dziedziczenia:

```

Entity (bazowa klasa dla wszystkich obiektów na planszy)
+-- Human (bazowa klasa dla ludzi)
|   +-- Worker    (podwójny ruch)
|   +-- Student   (50% ś odporności)
|   +-- Elder      (50% szansy na ruch)
|   +-- Doctor    (ruch + leczenie)
+-- Hospital (stacjonarna jednostka lecznicza)

```

3.2. Zastosowane wzorce projektowe

3.2.1. Wzorzec Strategii (Strategy)

Wirusy nie dziedziczą logiki rozprzestrzeniania — komponują ją poprzez interfejs `ISpreadLogic`. Każdy wirus zawiera instancję konkretnej strategii:

- Flu → `UniformSpreadLogic`
- Covid → `DistanceWeightedSpreadLogic`
- Rabies → `FocusedSpreadLogic`

Dzięki temu można dowolnie łączyć wirusy ze strategiami bez modyfikacji istniejących klas.

3.2.2. Metoda Szablonowa (Template Method)

Klasa abstrakcyjna `SpreadLogicBase` definiuje szkielet metody `Spread()`:

1. Iteracja po wszystkich zarażonych ludziach
2. Dla każdego źródła — wywołanie abstrakcyjnej metody `SpreadFromSource()`
3. Sprawdzenie śmiertelności — losowanie względem `virus.Mortality`

Klasy pochodne implementują jedynie `SpreadFromSource()`, co wymusza spójną strukturę przy jednoczesnej elastyczności.

3.2.3. Wzorzec Fabryki (Factory)

`SpreadLogicFactory` tworzy i cache’uje singletonowe instancje strategii:

```
1 public static ISpreadLogic Create(SpreadLogicType type)
2 {
3     return type switch
4     {
5         SpreadLogicType.Uniform => _uniform ??= new
        UniformSpreadLogic(),
6         SpreadLogicType.DistanceWeighted => _distance ??= new
        DistanceWeightedSpreadLogic(),
7         SpreadLogicType.Focused => _focused ??= new
        FocusedSpreadLogic(),
8         _ => throw new ArgumentException($"Unknown spread logic
        type: {type}")
9     };
10 }
```

3.2.4. Wstrzykiwanie Zależności (Dependency Injection)

Interfejs `IRandom` jest wstrzykiwany do wszystkich klas wymagających losowości (`Board`, `Human`, `SpreadLogicBase`). W produkcji używany jest `GameRandom.Create()` (brak ziarna), w testach `GameRandom.Create(seed)` (deterministyczne, powtarzalne wyniki).

3.2.5. Segregacja Interfejsów (Interface Segregation)

Zamiast jednego rozbudowanego interfejsu, projekt zawiera pięć wyspecjalizowanych

- `ITile` — symbol, opis, priorytet renderowania
- `IVirus` — właściwości wirusa, metoda rozprzestrzeniania
- `ISpreadLogic` — strategia rozprzestrzeniania
- `IHealingAbility` — zdolność leczenia
- `IRandom` — generator liczb losowych

3.3. Pętla symulacji

Główna pętla w `Program.cs` składa się z dwóch faz na każdy tick:

1. **Aktualizacja jednostek** — każda jednostka na planszy wywołuje `Update(Board)`. Ludzie poruszają się, lekarze i szpitale leczą.
2. **Rozprzestrzenianie wirusa** — aktywny wirus wywołuje `Spread(Board)`, który poprzez strategię rozprzestrzeniania infekuje nowe ofiary i sprawdza śmiertelność.

Warunki zakończenia symulacji:

- Liczba zarażonych osiąga 0 (wirus wyeliminowany)
- Wszyscy ludzie są zarażeni
- Liczba zarażonych nie zmienia się przez 100 kolejnych ticków (stagnacja)

4. Opis komponentów

4.1. Interfejsy (Interfaces/)

- `ITile` — definiuje `Symbol` (znak), `Description` (opis), `RenderPriority` (priorytet renderowania). Implementowany przez `Entity`.
- `IVirus` — definiuje `Name`, `Infectivity`, `Mortality`, `InfectionRange` oraz metodę `Spread(Board)`.
- `ISpreadLogic` — kontrakt dla strategii rozprzestrzeniania: `Spread(IVirus, Board)`.
- `IHealingAbility` — definiuje `Range`, `Effectiveness`, `Heal(Human)`. Implementowany przez `Doctor` i `Hospital`.
- `IRandom` — abstrakcja generatora liczb losowych: `Next()`, `Next(int)`, `Next(int, int)`, `NextSingle()`.

4.2. Jednostki (Models/Entities/)

4.2.1. Entity

Klasa bazowa dla wszystkich obiektów na planszy. Przechowuje współrzędne (x, y) , definiuje wirtualne właściwości `Symbol`, `Description`, `RenderPriority` oraz wirtualną metodę `Update(Board)`.

4.2.2. Hospital

Stacjonarna jednostka lecznicza (symbol +). Implementuje `IHealingAbility`:

- Zasięg: 2
- Skuteczność: 1.0
- Mechanizm: w każdej aktualizacji znajduje zarażonych w zasięgu i zwiększa ich licznik `HealingTicks`; po 5 tickach następuje pełne wyleczenie.

4.3. Ludzie (Models/Humans/)

4.3.1. Human

Klasa bazowa dla wszystkich typów ludzi. Kluczowe pola i metody:

- `IsAlive` — czy żyje
- `Virus` — wirus, którym jest zarażony (lub `null`)
- `IsInfected` — czy jest zarażony
- `HealingTicks` — licznik postępu leczenia
- `Age` — wiek (wpływa na migrację)
- `HomeRegion` — region macierzysty
- `MigrationCooldown` — czas do kolejnej migracji

Mechanika ruchu:

1. Ruch lokalny: o 1 krok w losowym kierunku ($\Delta x, \Delta y \in \{-1, 0, 1\}$)
2. Redukcja przez świadomość: ruch tylko gdy `rng.NextSingle() > AwarenessLevel`
3. Blokada: podczas lockdownu nie można opuścić swojego regionu
4. Migracja dalekodystansowa: losowa teleportacja do innego regionu z cooldownem 15 ticków

Prawdopodobieństwo migracji:

$$P_{\text{migracji}} = 0.03 \cdot M_{\text{zawodu}} \cdot \left(1.4 - \frac{\text{Age}}{100}\right)$$

gdzie M_{zawodu} wynosi: Student 1.4, Worker 1.1, Doctor 1.0, Elder 0.7. Mnożnik wieku jest clampowany do przedziału $[0.6, 1.4]$.

4.3.2. Worker

- Symbol: W
- Wiek: 23–61
- Specjalizacja: wykonuje **dwa ruchy** na tick (dwukrotne wywołanie `Move()`)

4.3.3. Student

- Symbol: S
- Wiek: 16–26
- Specjalizacja: **50% naturalnej odporności** — metoda `Infect()` ma 50% szansy na odrzucenie infekcji

4.3.4. Elder

- Symbol: 0
- Wiek: 65–91
- Specjalizacja: działa tylko w **50% aktualizacji** (pomija swoją turę gdy `rng.NextSingle() < 0.5f`)

4.3.5. Doctor

- Symbol: D
- Implementuje `IHealingAbility`: zasięg 3, skuteczność 0.8
- W każdej aktualizacji: najpierw się przemieszcza, następnie leczy zarażonych w zasięgu
- Leczenie trwa 5 ticków (addytywne z innymi źródłami leczenia)

4.4. Wirusy (Models/Viruses/)

Wirus	Infectivity	Mortality	Zasięg	Strategia
Flu	0.25	0.005 (0.5%)	1	Uniform
Covid	0.65	0.02 (2%)	2	DistanceWeighted
Rabies	0.50	0.30 (30%)	1	Focused

Tabela 1: Parametry wirusów

Każdy wirus deleguje metodę `Spread()` do swojej strategii rozprzestrzeniania (**wzorzec Strategii**).

4.5. Strategie rozprzestrzeniania (Models/SpreadLogic/)

4.5.1. SpreadLogicBase

Klasa abstrakcyjna implementująca **metodę szablonową**:

```

1 public void Spread(IVirus virus, Board board)
2 {
3     var infected = board.GetInfectedHumans();
4     foreach (var source in infected)
5     {
6         SpreadFromSource(virus, board, source);
7
8         // Sprawdzenie smiertelnosci
9         if (rng.NextSingle() <= virus.Mortality)
10            source.Die();
11     }
12 }
13
14 protected abstract void SpreadFromSource(IVirus virus, Board
    board, Human source);

```

4.5.2. UniformSpreadLogic

Każdy zdrowy człowiek w zasięgu ma taką samą szansę na zarażenie:

$$P_{\text{infekcji}} = \text{Infectivity}$$

4.5.3. DistanceWeightedSpreadLogic

Szansa na zarażenie maleje liniowo z odległością:

$$P_{\text{infekcji}} = \text{Infectivity} \cdot \left(1 - \frac{\text{distance}}{\text{range} + 1}\right)$$

gdzie distance to odległość euklidesowa (zaokrąglona w górę).

4.5.4. FocusedSpreadLogic

Wybierany jest jeden losowy cel w zasięgu, który otrzymuje szansę na zarażenie równą Infectivity.

4.6. Plansza i generowanie mapy

4.6.1. Board

Centralny stan gry. Zawiera:

- Dwuwymiarową tablicę kafelków `Tile[,]` grid o rozmiarze `40×40`
- Listę jednostek `List<Entity> entities`
- Właściwości: `AwarenessLevel`, `LockdownEnabled`

Kluczowe metody:

- `IsWalkable(int x, int y)` — sprawdza czy kafelek jest przechodni
- `GetHumansInRange(int cx, int cy, int range)` — znajdowanie ludzi w zasięgu kołowym (wykorzystuje kwadrat odległości euklidesowej)
- `GetInfectedHumans()` — zwraca listę zarażonych, żywych ludzi
- `addRandomEntity(Func<int,int,Entity> factory)` — umieszcza jednostkę na losowym kafelku łądowym

4.6.2. MapGenerator

Proceduralne generowanie mapy z wykorzystaniem algorytmu **Voronoia** z **BFS**:

1. Wybór 4 punktów zarodkowych (seed points) z minimalnym odstępem $\text{regionRadius} \cdot 2 \cdot 0.5$
2. Ekspansja BFS z użyciem `PriorityQueue` (priorytetem jest odległość)
3. Strefa buforowa wokół granic regionów zapobiega ich bezpośredniemu sąsiedztwu
4. Nieprzypisane komórki stają się wodą (`Water`)
5. Każdy region otrzymuje unikalną wartość enum (`Region0-Region3`)

4.7. Renderowanie

Klasa `Rendering` odpowiada za wyświetlanie stanu symulacji w konsoli z użyciem kolorów ANSI:

- Zarażeni ludzie — kolor czerwony
- Martwi ludzie — kolor czarny
- Lekarze — jasny zielony
- Szpitale — żółty
- Woda — jasny niebieski
- Łąd — domyślny

Każda komórka renderuje symbol jednostki (jeśli występuje) lub symbol kafelka.

4.8. Statystyki

Klasa `Statistics` kolekcjonuje dane w postaci krotek (`Tick`, `Infected`, `Dead`) i eksportuje je do pliku `simulation.csv` po zakończeniu symulacji.

5. Mechanizmy symulacyjne

5.1. Świadomość społeczna (Awareness)

W miarę wzrostu liczby zarażonych, ludzie zmniejszają swoją mobilność:

$$\text{AwarenessLevel} = \min\left(1, \frac{\text{liczba zarażonych}}{300}\right)$$

Ruch następuje tylko gdy `rng.NextSingle() > AwarenessLevel`. Przy 300 lub więcej zarażonych świadomość osiąga maksimum (1.0), całkowicie zatrzymując dobrowolny ruch.

5.2. Blokada (Lockdown)

Aktywuje się gdy liczba zarażonych przekracza 80% żyjącej populacji. Podczas lockdownu ludzie nie mogą przekraczać granic swojego regionu macierzystego.

5.3. System leczenia

Zarówno lekarze, jak i szpitale implementują `IHealingAbility`:

- Każde źródło leczenia zwiększa `HealingTicks` zarażonego o 1 na tick
- Po osiągnięciu 5 ticków następuje całkowite wyleczenie (`Heal()`)
- Wiele źródeł leczenia działa addytywnie — np. lekarz i szpital razem leczą w 2.5 ticka

Jednostka	Zasięg	Skuteczność
Doctor	3	0.8
Hospital	2	1.0

Tabela 2: Parametry jednostek leczących

6. Testy

Projekt zawiera zestaw testów jednostkowych w technologii xUnit (`tests/VirusSimu`

6.1. Konfiguracja

Testy wykorzystują `GameRandom.Create(42)` (deterministyczne ziarno) dla zapewnienia powtarzalności. Równoległe wykonywanie testów jest wyłączone (`xunit.runner.json`).

6.2. Pokrycie testów

- **HumanTests:** infekcja, leczenie, śmierć, odporność Studentów (test statystyczny na 2000 prób), podwójny ruch Workera, 50% ruchu Eldera (1000 prób)
- **BoardTests:** przechodniość kafelków, pobieranie jednostek, zakresy, blokada regionów, losowe rozmieszczanie
- **EntityTests:** leczenie przez szpital (stopniowe i pełne), leczenie przez lekarza, właściwości jednostek, SpreadLogicFactory
- **VirusTests:** właściwości wirusów, rozprzestrzenianie na pustej planszy, wszystkie trzy strategie (ziarno 42)
- **StatisticsTests:** format CSV, wiele wierszy, wiersze zerowe

6.3. Uruchamianie

```
dotnet test tests/VirusSimulation.Tests
```

7. Instrukcja uruchomienia

7.1. Wymagania

- .NET SDK 10.0 lub nowszy

7.2. Budowanie

```
dotnet build
```

7.3. Uruchamianie

```
dotnet run
```

Program pyta o:

1. Typ wirusa (Flu / Covid / Rabies)
2. Liczbę poszczególnych jednostek (Workers, Students, Elders, Doctors, Hospitals)

7.4. Przebieg symulacji

1. Generowanie mapy (40×40)
2. Rozmieszczenie jednostek na lądzie
3. Zarażenie pierwszej napotkanej osoby

4. Pętla ticków (z opóźnieniem 200 ms):
 - Aktualizacja jednostek
 - Rozprzestrzenianie wirusa
 - Renderowanie stanu
 - Zapis statystyk
5. Zakończenie po spełnieniu warunku brzegowego
6. Eksport wyników do `simulation.csv`

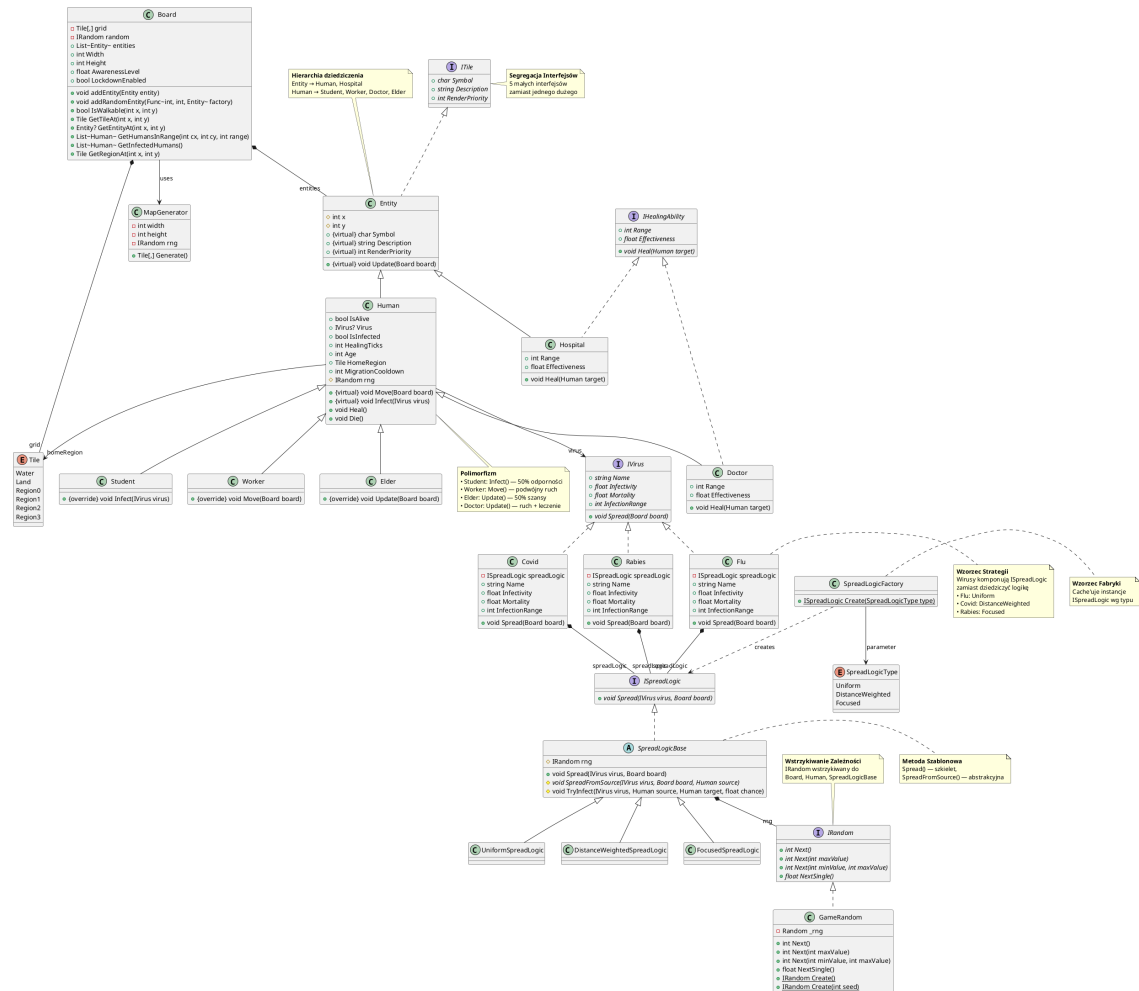
8. Podsumowanie

Projekt `VirusSimulation` stanowi kompleksową implementację symulacji epidemii w języku C# z wykorzystaniem zaawansowanych mechanizmów programowania obiektowego:

- **Czytelna architektura** — hierarchia dziedziczenia, pięć wyspecjalizowanych interfejsów
- **Wzorce projektowe** — Strategia, Metoda Szablonowa, Fabryka, Wstrzykiwanie Zależności, Segregacja Interfejsów
- **Testowalność** — deterministyczne ziarno RNG, interfejs `IRandom`, wysoka izolacja komponentów
- **Rozszerzalność** — łatwość dodawania nowych typów wirusów, strategii rozprzestrzeniania, jednostek i mechanik
- **Proceduralna generacja** — algorytm Voronoia z BFS do tworzenia zróżnicowanych map
- **Złożone zachowania** — migracja, świadomość, blokada, system leczenia, odporność, statystyki

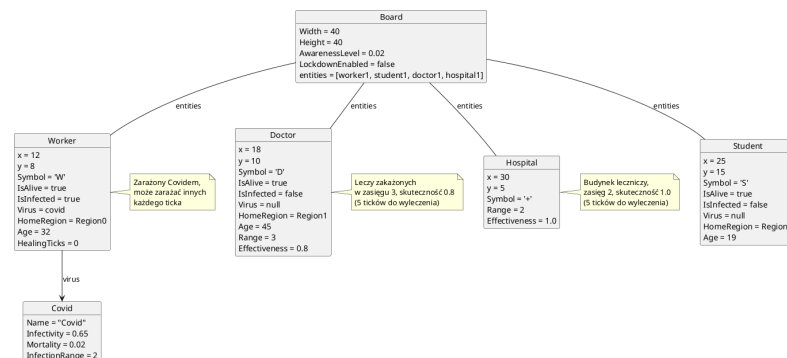
A. Diagramy UML

A.1. Diagram klas



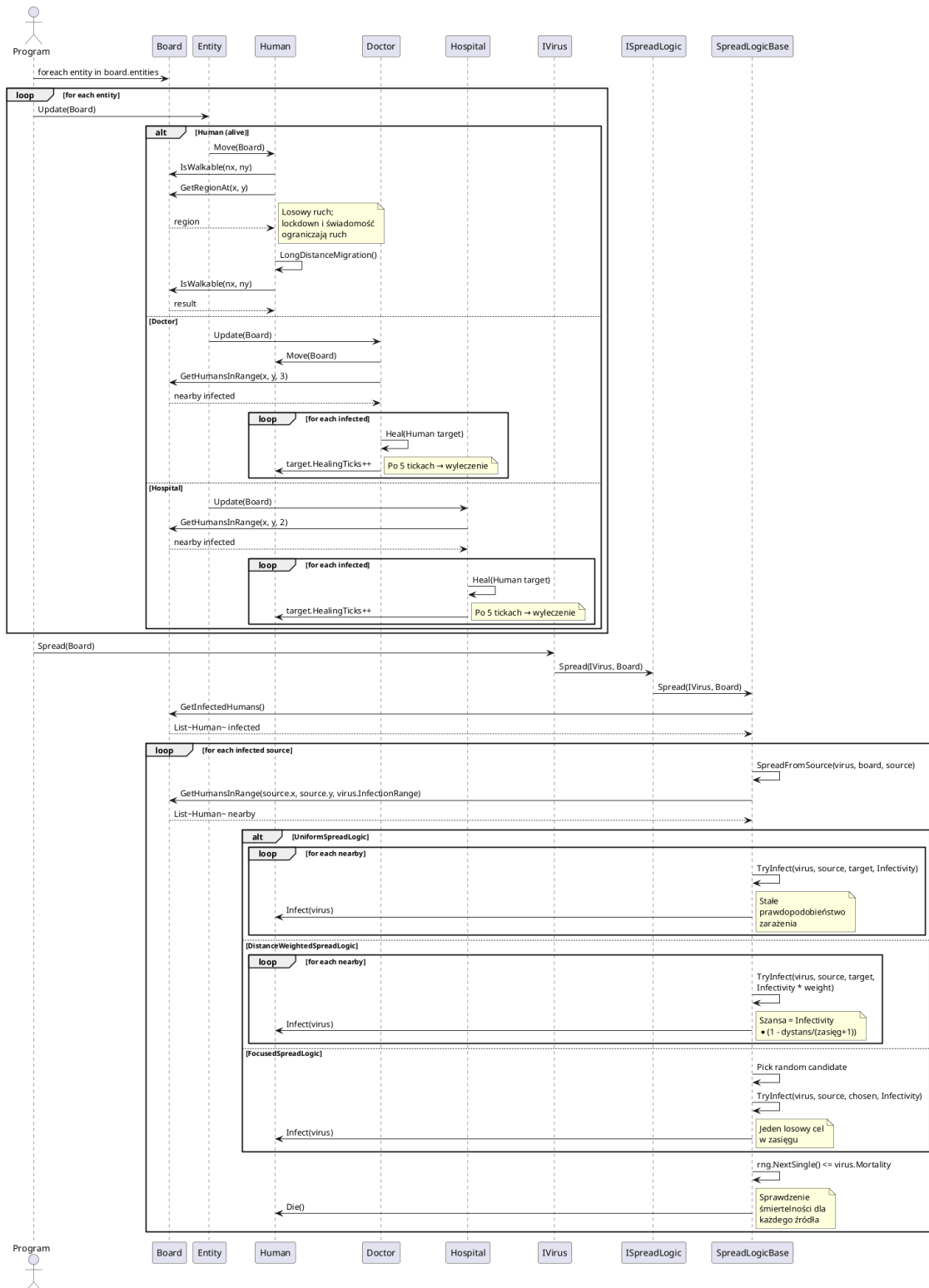
Rysunek 1: Diagram klas — hierarchia dziedziczenia, realizacje interfejsów, kompozycje

A.2. Diagram obiektów



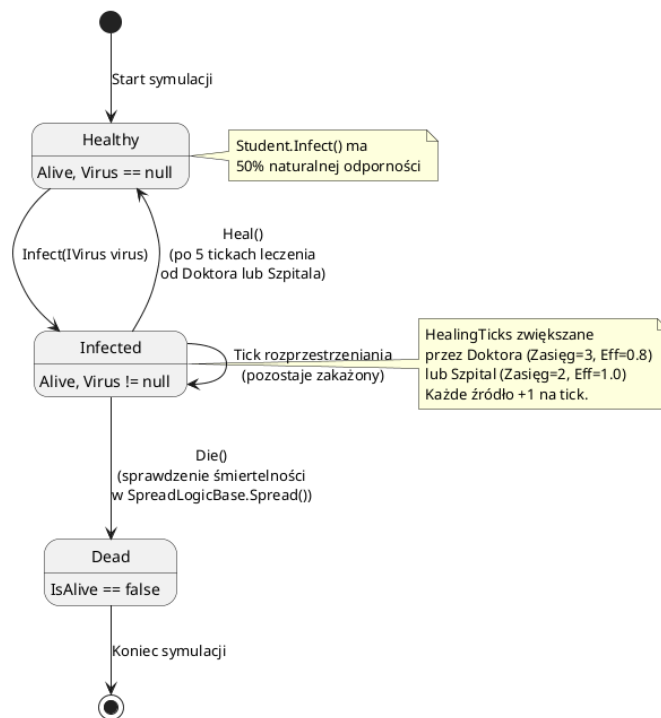
Rysunek 2: Diagram obiektów — przykładowy stan symulacji

A.3. Diagram sekwencji



Rysunek 3: Diagram sekwencji — dwufazowa pętla symulacji

A.4. Diagram stanów



Rysunek 4: Diagram stanów człowieka — cykl życia